

PRACTICAL 4



CHARUSAT[®]

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

FACULTY OF TECHNOLOGY AND ENGINEERING

DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY

AND RESEARCH

DEPARTMENT OF COMPUTER ENGINEERING

A.Y 2026 – 27 Semester – 3

LAB MANUAL

CSUC201 Fundamentals of Data Structures and Algorithms



PRACTICAL 4

Practical 4:

Singly Linked List: Dynamic Insertion, Deletion, and Traversal.

Practical Aim:

To implement core singly linked list operations and develop the ability to manipulate dynamic sequences through real-world problem scenarios.

Problem 1:

A hospital manages a queue of patient tokens. New critical patients must be added to the front, routine patients are added to the end, and occasionally a patient with a priority number must be inserted at a specific position in the queue. Given a sequence of such operations, implement all three insertion types and print the final queue after each operation.

Describe how your insertion at a specific position works. What does your solution do if the position given is greater than the current length of the queue? Is that handled explicitly or does it cause an error?

Theory:

Inserting at a position beyond the current length of the list is an invalid operation. Should your solution silently insert at the end, report an error, or do something else entirely? Justify your choice and explain what assumption about the caller your choice makes.

Inserting at a position beyond the current length of the list should be treated as an invalid operation, and the program should report an error rather than silently inserting the element at the end. This makes the behavior clear and prevents unexpected changes to the queue. It assumes that the caller provides a valid position within the current list range and that any position greater than the list length is considered an input error that should be handled explicitly.

Code:

https://github.com/kanan2007s-sys/FDSA_B/tree/main/Practical4

PRACTICAL 4

Screenshot/Output:

```
1. Critical
2. Routine
3. Insert at Position
0. Exit
Enter choice: 1
Enter patient name: kanan
kanan

1. Critical
2. Routine
3. Insert at Position
0. Exit
Enter choice: 2
Enter patient name: tanay
kanan tanay

1. Critical
2. Routine
3. Insert at Position
0. Exit
Enter choice: 3
Enter patient name: someone
Enter position: 2
kanan someone tanay

1. Critical
2. Routine
3. Insert at Position
0. Exit
Enter choice: 0

Process returned 0 (0x0)   execution time : 49.206 s
Press any key to continue.
```

Description:

In this problem we have to use the concept of Singly linked list. In a hospital from the level of critical condition position of the patient is assigned.

Challenges:

Trying in the first attempt was very confusing . Couldn't understand where to put the pointer, where to declare head , where to declare temp variable etc. Struct was better or class concept to implement linked list.

Conclusion:

The first problem successfully implements the three main insertion operations in a singly linked list: insertion at the front, insertion at the end, and insertion at a specific position. It also handles invalid positions explicitly instead of causing an error or silently inserting at the end. Through this implementation, the dynamic nature of linked lists and manipulation of node links are demonstrated effectively.

PRACTICAL 4

Problem 2:

The hospital also needs to remove a patient token from the queue when a patient leaves, print all remaining tokens from last to first for an end-of-day audit, and display the full queue from front to back at any point. Implement deletion by value, reverse printing, and forward traversal on the same queue from Problem 4.1a.

Describe how your reverse printing works. Does it use any extra storage, or does it work purely by following the links? Can you think of another way to reverse-print without modifying the original queue?

Theory:

If the node to be deleted is the only node in the list, how many pointers need to change and what must they be set to? How is this case structurally different from deleting a node in the middle?

If the node to be deleted is the only node in the list, only one pointer needs to change: head. It must be set to nullptr, because after deleting that node, the list becomes empty. For example, if head -> A -> nullptr, after deletion, head = nullptr. In contrast, when deleting a node from the middle, the next pointer of the previous node must be changed to point to the node after the deleted node. For example, A -> B -> C becomes A -> C. Thus, deleting the only node removes the entire list, while deleting a middle node connects the remaining nodes together.

Code:

https://github.com/kanan2007s-sys/FDSA_B/tree/main/Practical4

PRACTICAL 4

Output/Screenshot:

```
1. Critical
2. Routine
3. Insert at Position
4. Delete
5. Forward Display
6. Reverse Display
0. Exit
Enter choice: 1
Enter patient name: Kanan
Front to Back: Kanan
```

```
1. Critical
2. Routine
3. Insert at Position
4. Delete
5. Forward Display
6. Reverse Display
0. Exit
Enter choice: 1
Enter patient name: someone
Front to Back: someone Kanan
```

```
1. Critical
2. Routine
3. Insert at Position
4. Delete
5. Forward Display
6. Reverse Display
0. Exit
Enter choice: 1
Enter patient name: anotherone
Front to Back: anotherone someone Kanan
```

```
1. Critical
2. Routine
3. Insert at Position
4. Delete
5. Forward Display
6. Reverse Display
0. Exit
Enter choice: 4
Enter patient name to delete: someone
Front to Back: anotherone Kanan
```

```
1. Critical
2. Routine
3. Insert at Position
4. Delete
5. Forward Display
6. Reverse Display
0. Exit
Enter choice: 5
Front to Back: anotherone Kanan
```

```
1. Critical
2. Routine
3. Insert at Position
4. Delete
5. Forward Display
6. Reverse Display
0. Exit
Enter choice: 6
Back to Front: Kanan anotherone
```

```
1. Critical
2. Routine
3. Insert at Position
4. Delete
5. Forward Display
6. Reverse Display
0. Exit
Enter choice: 0
Exiting...
```

Description:

From practical 1 we have to add three more functionalities to print patients in reverse , delete a patient , or traversal.

PRACTICAL 4

Challenge:

Again the main challenge was to use and create pointer . How next pointer will be used and etc.

Github repository link:

https://github.com/kanan2007s-sys/FDSA_B

Conclusion:

The second problem successfully implements the main deletion and traversal operations of a singly linked list. Deletion by value removes the required patient while correctly maintaining the links between the remaining nodes. Forward traversal displays the queue from front to back, while reverse printing displays the patients from last to first without modifying the original list. Overall, this problem demonstrates how nodes can be safely removed and how linked-list data can be accessed in both forward and reverse order.